# Summary Problem 3 – Nicola Dall'Acqua

## Fully Connected Neural Network

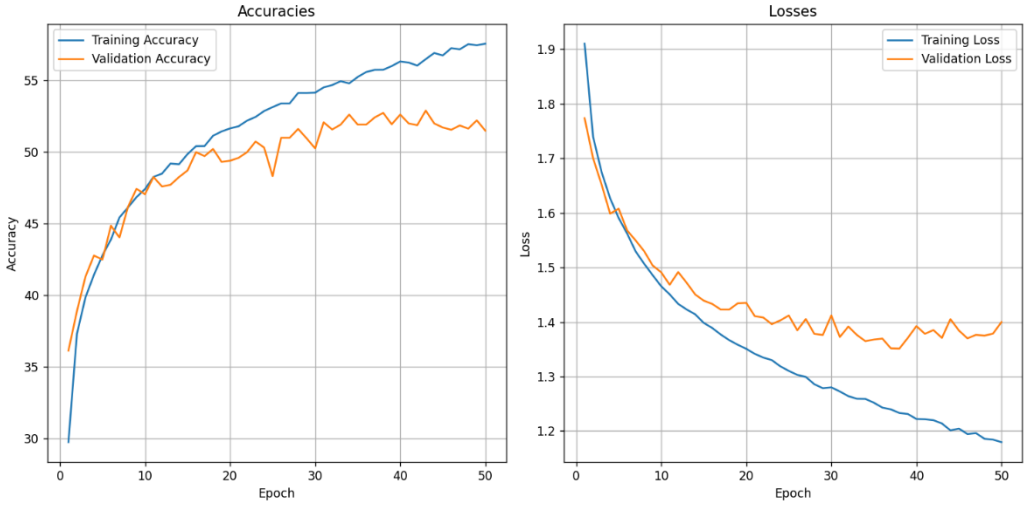
### Model

```
def __init__(self):
  super(FCNN, self).__init__()
  self.fc1 = nn.Linear(32 * 32 * 3, 512)
  self.fc2 = nn.Linear(512, 512)
  self.fc3 = nn.Linear(512, 256)
  self.fc4 = nn.Linear(256, 256)
  self.fc5 = nn.Linear(256, 128)
  self.fc6 = nn.Linear(128, 128)
  self.fc7 = nn.Linear(128, 10)
def forward(self, x):
  x = x.view(-1, 32 * 32 * 3)
  x = F.relu(self.fc1(x))
  x = F.relu(self.fc2(x))
  x = F.relu(self.fc3(x))
  x = F.relu(self.fc4(x))
  x = F.relu(self.fc5(x))
  x = F.relu(self.fc6(x))
  return self.fc7(x)
```

### Summary

- 7 fully connected linear layers (sizes as above)
- ReLU as activation function
- Cross Entropy Loss
- Training for 50 Epochs
- Optimizer: optimizer = torch.optim.Adam(self.parameters(), lr=lr, weight_decay=1e-4)

## Accuracy & Error Plots




## Final Test Accuracy/Loss

- Final test accuracy of FCNN: **53.42%**, Final test loss of FCNN: 1.319835966146445

**Training time:** ~40 Minutes

# Convolutional Neural Network

## Model

```
def __init__(self):
  super(CNN, self).__init__()
  self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)  # 32×32×32
  self.bn1 = nn.BatchNorm2d(32)
  self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)  # 64×32×32
  self.bn2 = nn.BatchNorm2d(64)
  self.pool1 = nn.MaxPool2d(2, 2)  # 64×16×16
  self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)  # 128×16×16
  self.bn3 = nn.BatchNorm2d(128)
  self.se1 = SEBlock(128)
  self.conv4 = nn.Conv2d(128, 256, kernel_size=3, padding=1)  # 256×16×16
  self.bn4 = nn.BatchNorm2d(256)
  self.pool2 = nn.MaxPool2d(2, 2)  # 256×8×8
  self.conv5 = nn.Conv2d(256, 256, kernel_size=3, padding=1)  # 256×8×8
  self.bn5 = nn.BatchNorm2d(256)
  self.conv6 = nn.Conv2d(256, 128, kernel_size=3, padding=1)  # 128×8×8
  self.bn6 = nn.BatchNorm2d(128)
  self.se2 = SEBlock(128)
  self.conv7 = nn.Conv2d(128, 128, kernel_size=3, padding=1)  # 128×8×8
  self.bn7 = nn.BatchNorm2d(128)
  self.conv8 = nn.Conv2d(128, 128, kernel_size=3, padding=1)  # 128×8×8
  self.bn8 = nn.BatchNorm2d(128)
  self.conv9 = nn.Conv2d(128, 128, kernel_size=3, padding=1)  # 128×8×8
  self.bn9 = nn.BatchNorm2d(128)
```

```
    self.se3 = SEBlock(128)
    self.pool3 = nn.MaxPool2d(2, 2)  # 128×4×4
    self.dropout = nn.Dropout(0.5)
    self.fc1 = nn.Linear(128, 128)
    self.fc2 = nn.Linear(128, 10)

def forward(self, x):
    x = F.relu(self.bn1(self.conv1(x)))
    x = F.relu(self.bn2(self.conv2(x)))
    x = self.pool1(x)
    x = F.relu(self.bn3(self.conv3(x)))
    x = self.se1(x)
    x = self.bn4(self.conv4(x))
    x_res1 = self.pool2(x)
    x = self.bn5(self.conv5(x_res1))
    x = F.relu(x + x_res1)
    x = self.bn6(self.conv6(x))
    x_res2 = self.se2(x)
    x = self.bn7(self.conv7(x_res2))
    x = F.relu(x + x_res2)
    x_res3 = self.bn8(self.conv8(x))
    x = self.bn9(self.conv9(x_res3))
    x = self.se3(x)
    x = F.relu(x + x_res3)
    x = self.pool3(x)
    x = F.adaptive_avg_pool2d(x, 1)
    x = x.view(x.size(0), -1)
    x = F.relu(self.dropout(self.fc1(x)))
    x = self.fc2(x)
    return x
```

## Explanation of SEBlocks

Squeeze-and-Excitation Blocks are add an attention mechanism that:

- Learns to keep important information
- Supress non-important information

They consist of:

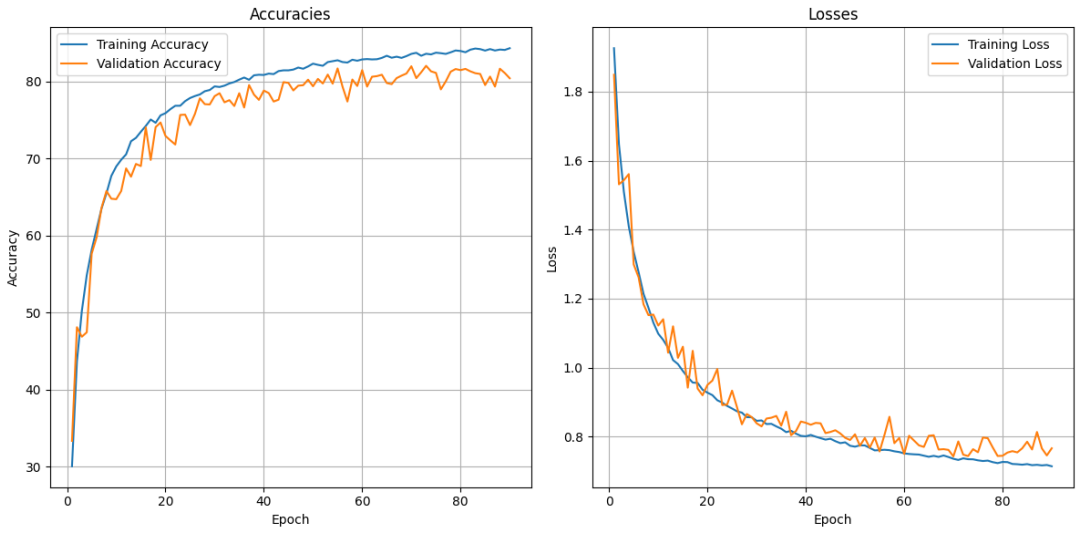
```
self.fc = nn.Sequential(
    nn.AdaptiveAvgPool2d(1),  # Squeeze
    nn.Flatten(),          # Shape: (B, C)
    nn.Linear(channel, channel // 16, bias=False),
    nn.ReLU(inplace=True),
    nn.Linear(channel // 16, channel, bias=False),
    nn.Sigmoid()
)
```

Where “channel” is the input number passed in the forward-function of the model (i.e. 128). By dividing by 16, the model must learn to represent information in a more compact way.

## Summary

- 9 Convolutional Layers with kernel size 3 and padding 1
- 9 Batch Norm Layers (keep the input size)
- 3 Max-Pooling Layers with Pool size 2x2
- 3 SEBlocks (with 2 fully connected layers each and Average Pooling)
- 2 fully connected classifier layers with Dropout applied to the first of these
- Residual connections from Conv-Layer 4 to Conv-Layer 5, from Conv-Layer 6 to Conv-Layer 7 and from Conv-Layer 8 to Conv-Layer 9 (These help with gradient flow)
- The output sizes of all layers in order without BatchNorms are: (the total sizes can be seen in the model above)
    - Conv1: 32 x 32 x 32
    - Conv2: 64 x 32 x 32
    - Pool1: 64 x 16 x 16
    - Conv3: 128 x 16 x 16
    - SE1: 128 → 8 & 8 → 128
    - Conv4: 256 x 16 x 16
    - Pool2: 256 x 8 x 8
    - Conv5: 256 x 8 x 8
    - Conv6: 128 x 8 x 8
    - SE2: 128 → 8 & 8 → 128
    - Conv7: 128 x 8 x 8
    - Conv8: 128 x 8 x 8
    - Conv9: 128 x 8 x 8
    - SE3: 128 → 8 & 8 → 128
    - Pool3: 128 x 4 x 4
    - FC1: 128 → 128 with dropout (probability 0.5)
    - FC2: 128 → 10
- Cross Entropy Loss
- Training for 90 epochs
- Optimizer: Adam with scheduled Cosine Annealing

## Accuracy & Error Plots



## Final Test Accuracy/Loss

- Final test accuracy of CNN: **90.33%**, Final test loss of CNN: 0.541554115241087

**Training time:** ~3 hours

# Recommended Model

I recommend the CNN model, as the CNN's losses are much lower than the FCNN's and the CNN's accuracies much higher than the FCNN's.

Training the CNN does take longer than training the FCNN, but this additional computational effort is more than worth it for the increase in performance. This is especially visible in the final test accuracies; the CNN has **90.33%** which is much higher than the FCNN's **53.42%**.

It may be possible to slightly increase the performance of the FCNN by adding some more layers/parameters and a stronger regularization, but from my experiments, this would lead to an increase in the final test accuracy of only a few percents, so ultimately, the CNN would still be a lot better.

## Side note: Validation Accuracies vs. Test Accuracies

As is visible above, the validation accuracies of both models are quite a bit lower than their respective test accuracies. The reason for that lies in my data augmentation:

```
train_transform = transforms.Compose([
  transforms.RandomHorizontalFlip(p=0.5),
  transforms.RandomRotation(20),
  transforms.ColorJitter(brightness = 0.1,contrast = 0.1,saturation = 0.1),
  transforms.RandomAdjustSharpness(sharpness_factor = 2,p = 0.2),
  transforms.AutoAugment(policy=AutoAugmentPolicy.CIFAR10),
  transforms.ToTensor(),
  transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
  transforms.RandomErasing(p=0.3,scale=(0.02, 0.1),value=1.0, inplace=False)
])
test_transform = transforms.Compose([
  transforms.ToTensor(),
  transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

"train_transform" is applied to the train and validation datasets, "test_transform" to the test dataset. This essentially regularizes the model as whenever it is classifying a sample from the validation set, it must solve a more difficult task (account for all the augmentation). This explains why the validation accuracy is lower than the test accuracy, as the model is doing something more difficult.